

Programação Funcional

Aula 05 - Funções de Alta Ordem

Eliseu C. Miguel

Universidade Federal de Alfenas

9 de junho de 2026

Tópicos

- 1 Introdução às Funções de Alta Ordem
- 2 Função Map
- 3 Função Filter
- 4 Função Fold
- 5 Exercícios
- 6 Referências

Tópicos

- 1 Introdução às Funções de Alta Ordem
- 2 Função Map
- 3 Função Filter
- 4 Função Fold
- 5 Exercícios
- 6 Referências

Tópicos

- 1 Introdução às Funções de Alta Ordem
- 2 Função Map
- 3 Função Filter
- 4 Função Fold
- 5 Exercícios
- 6 Referências

Tópicos

- 1 Introdução às Funções de Alta Ordem
- 2 Função Map
- 3 Função Filter
- 4 Função Fold
- 5 Exercícios
- 6 Referências

Tópicos

- 1 Introdução às Funções de Alta Ordem
- 2 Função Map
- 3 Função Filter
- 4 Função Fold
- 5 Exercícios
- 6 Referências

Tópicos

- 1 Introdução às Funções de Alta Ordem
- 2 Função Map
- 3 Função Filter
- 4 Função Fold
- 5 Exercícios
- 6 Referências

O que são?

- **Funções de Alta Ordem** (Higher-Order Functions) são funções que:
 - Recebem uma ou mais funções como argumento.
 - E/ou retornam uma função como resultado.
- Elas nos permitem criar abstrações poderosas e reutilizáveis.

Exemplo Conceitual

```
operarSobreLista :: (a -> b) -> [a] -> [b]
```

Esta função recebe outra função $(a \rightarrow b)$ como seu primeiro argumento.

Função map

- map aplica uma função a **cada elemento** de uma lista, retornando uma nova lista com os resultados.

Assinatura

```
map :: (a -> b) -> [a] -> [b]
```

Exemplos

```
map (+1) [1, 2, 3] = [2, 3, 4]
```

```
map even [1, 2, 3] = [False, True, False]
```

```
map reverse ["ola", "amor"] = ["alo", "roma"]
```

Função map

Sem map

```
double :: [Int] -> [Int]
double [] = []
double (x:xs) = (2*x) : double xs
```

Com map

```
double xs = map (*2) xs
```

A Função filter

- `filter` seleciona os elementos de uma lista que satisfazem um **predicado** (uma função que retorna `Bool`).
- Retorna uma nova lista contendo apenas os elementos para os quais o predicado foi `True`.

Assinatura

```
filter :: (a -> Bool) -> [a] -> [a]
```

Exemplos

```
filter even [1, 2, 3, 4, 5] = [2, 4]
```

```
filter (>3) [1, 5, 2, 8] = [5, 8]
```

```
filter (<3) [1, 5, 2, 8] = [1, 2]
```

Valor acumulado em Listas: Fold ($\neq$ no Prelude)

- As funções fold (dobrar) são usadas para combinar os elementos de uma lista em um único valor.

fold

```
fold :: (u -> t -> u) -> u -> [t] -> u  
fold f s [] = s  
fold1 f s (a:x) = f a (fold1 f s x)
```

Exemplo de execução

```
fold (+) 0 [1, 2, 3]  
= 1 + (2 + (3 + 0)) = 6
```

Valor acumulado em Listas: Fold ($\nexists$ no Prelude)

Funções que são implementadas com fold

```
sum xs = fold (+) 0 xs
```

```
product xs = fold (*) 1 xs
```

Foldr: Prelude

- foldr (fold right) processa uma lista da **direita para a esquerda**.
- O acumulador é o último argumento da função de combinação.

Assinatura no Prelude

```
foldr :: (t -> u -> u) -> u -> [t] -> u  
foldr f s [] = s  
foldr f s (a:x) = f a (foldr f s x)
```

Foldl: Prelude

- `foldl` (fold left) processa uma lista da **esquerda para a direita**.
- O acumulador é o primeiro argumento da função de combinação.

Assinatura no Prelude

```
foldl :: (u -> t -> u) -> u -> [t] -> u  
foldl f s [] = s  
foldl f s (a:x) = foldl f (f s a) x
```

Exemplos de Execução

foldl

```
foldl (+) 0 [1, 2, 3]
= foldl (+) (0 + 1) [2, 3]
= foldl (+) (1 + 2) [3]
= foldl (+) (3 + 3) []
= 6
```

foldr

```
foldr (+) 0 [1, 2, 3]
= 1 + (foldr (+) 0 [2, 3])
= 1 + (2 + (foldr (+) 0 [3]))
= 1 + (2 + (3 + (foldr (+) 0 [])))
= 1 + (2 + (3 + 0)) = 6
```


Exercícios

- 1 Use `map` para criar uma função `quadrados :: [Int] -> [Int]` que retorna o quadrado de cada número em uma lista.
- 2 Use `filter` para criar uma função `multiplosDe5 :: [Int] -> [Int]` que retorna apenas os números que são múltiplos de 5.
- 3 Use `map` e `filter` para criar uma função `quadradoDosPares :: [Int] -> [Int]` que retorna o quadrado apenas dos números pares de uma lista.
- 4 Re-implemente a função `concat'` que concatena uma lista de listas usando `foldr`. `concat' [[1,2],[3],[],[4,5]] = [1,2,3,4,5]`.
- 5 Use `foldl` para implementar a função `reverse'` que inverte uma lista.

Referências Bibliográficas

Slides [1]

Livro [2]



Vladimir Oliveira Di Iorio.

Programação Funcional - Tipos Básicos e Programas Simples em Haskell.

DPI - UFV.



Miran Lipovaca.

Learn You a Haskell for Great Good!

Lipovaca.

Sobre as apresentações didáticas

Atenção: este material não é completo para seus estudos.
Ao contrário, é apenas um guia para lhe informar quais são os tópicos importantes a serem estudados.

O estudo completo e necessário dá-se por materiais específicos sobre cada tópico, como livros e conteúdo na Internet.

Selecione materiais de fontes de seu interesse considerando a bibliografia citada no plano de ensino da disciplina para estudar os tópicos aqui abordados.

Consulte o professor, em caso de dúvidas

Agradecimentos especiais

Agradeço ao monitor da disciplina Felipe Araújo Correia por ter elaborado esta apresentação.

Agradeço a toda a comunidade $\text{\LaTeX}$.
Em especial a *Till Tantau* pelo *Beamer*.

<https://www.tcs.uni-luebeck.de/mitarbeiter/tantau/>

Desta forma, tornou-se possível a escrita deste material didático.